

Agentic Design Patterns

Revision cheat sheet for the course assignments: reflection, tool use, planning, and multi-agent systems.
Focus: what each pattern is for, why it helps, and reusable Python/LangChain boilerplates.

Core mental model: a useful agent is usually a loop around an LLM. It gathers context, applies a task-specific pattern, calls tools if needed, validates or refines the result, and returns an actionable output.

Pattern	Best For	Main Benefit
Reflection	Critique, revision, quality improvement	Improves first drafts through iterative self-review
Tool/API Use	Live data, external systems, dynamic answers	Grounds the model in real-world data
Planning	Specific goals with constraints and steps	Turns vague goals into executable workflows
Multi-Agent	Complex workflows with separable responsibilities	Improves modularity, clarity, and extensibility

1. Reflection Pattern

The agent generates an initial answer, critiques it, then revises it. This is useful when quality improves through feedback loops.

Use When

- You need a better answer than the first model response.
- The task involves critique, code review, analysis, or refinement.
- You want visible improvement across iterations.

Benefits

- Encourages clearer, deeper, and more feasible outputs.
- Makes weaknesses explicit before revision.
- Good for deliverables that need an output log or reflection history.

Watch For

- Can become repetitive if the critique prompt is vague.
- Costs more tokens because each iteration calls the model again.
- Needs a stopping rule: fixed rounds, score threshold, or user approval.

Boilerplate

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4", temperature=0.7)

critique_prompt = ChatPromptTemplate.from_messages([
    ("system", "Critique the draft and suggest practical fixes."),
    ("human", "Draft:\n{draft}")
])

revise_prompt = ChatPromptTemplate.from_messages([
    ("system", "Revise the draft using the critique."),
    ("human", "Draft:\n{draft}\n\nCritique:\n{critique}")
])

def reflective_agent(draft: str, rounds: int = 2) -> str:
    for _ in range(rounds):
        critique = llm.invoke(critique_prompt.format(draft=draft)).content
        draft = llm.invoke(revise_prompt.format(
            draft=draft, critique=critique
        )).content
    return draft
```

2. Tool/API Use Pattern

The agent calls an external tool first, then uses the model to interpret the result. This prevents the model from guessing current facts.

Use When

- The answer depends on live or external data.
- You need weather, stocks, files, databases, APIs, or calculations.
- The model should explain data rather than invent it.

Benefits

- Grounds outputs in real data.
- Separates retrieval from interpretation.
- Makes failures easier to report when the API returns an error.

Watch For

- API keys must be loaded safely from environment variables.
- Raw JSON should be summarized before sending to the model.
- Handle timeouts, missing fields, and rate limits.

Boilerplate

```
import os, requests
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model=os.getenv("OPENAI_MODEL", "gpt-4"))
prompt = ChatPromptTemplate.from_messages([
    ("system", "Explain the API data and recommend actions."),
    ("human", "Data summary:\n{summary}")
])

def fetch_data(location: str) -> dict:
    r = requests.get("https://api.example.com/data", params={
        "q": location, "apikey": os.getenv("API_KEY")
    }, timeout=20)
    r.raise_for_status()
    return r.json()

def tool_using_agent(location: str) -> str:
    raw = fetch_data(location)
    summary = f"temp={raw['main']['temp']}; condition={raw['weather'][0]['description']}"
    return llm.invoke(prompt.format(summary=summary)).content
```

3. Planning Pattern

The agent collects a clear objective and structured parameters, then decomposes the goal into phases, steps, checklists, or instructions.

Use When

- The task has a specific goal and constraints.
- The user needs an executable plan, not just an explanation.
- The output should include steps, timelines, lists, risks, or success criteria.

Benefits

- Reduces vague responses by giving the model boundaries.
- Turns broad goals into actionable tasks.
- Works well for most specific-purpose agents, not just event or meal planning.

Watch For

- Too much structure can make casual chat feel rigid.
- Missing constraints can cause impractical plans.
- Complex plans may need progress tracking or revision.

Boilerplate

```
from dataclasses import dataclass
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

@dataclass
class PlanRequest:
    objective: str
    constraints: str
    preferences: str
    output_format: str

prompt = ChatPromptTemplate.from_messages([
    ("system", "Break the objective into actionable steps."),
    ("human", "Objective: {objective}\nConstraints: {constraints}\n"
              "Preferences: {preferences}\nReturn: {output_format}")
])

def planning_agent(request: PlanRequest) -> str:
    return ChatOpenAI(model="gpt-4").invoke(
        prompt.format(**request.__dict__)
    ).content
```

4. Multi-Agent Pattern

The workflow is split between specialist agents. Each agent owns a specific responsibility, and an orchestrator coordinates the handoff.

Use When

- The task naturally has separate stages.
- Different parts need different tools or prompts.
- You want easier debugging and extension.

Benefits

- Improves clarity through division of labor.
- Makes each part easier to test independently.
- Lets you add new agents, such as visualization or alerts, without rewriting everything.

Watch For

- Agent boundaries should be real, not artificial.
- Handoffs need clean data structures.
- The orchestrator should make the workflow order obvious.

Boilerplate

```
class DataGatheringAgent:
    def run(self, query: str) -> dict:
        return fetch_external_data(query)

class DataProcessingAgent:
    def summarize(self, raw: dict) -> dict:
        return {"metric": raw["value"], "trend": "up"}
    def generate_insights(self, summary: dict) -> str:
        return llm.invoke(insight_prompt.format(summary=summary)).content

class VisualizationAgent:
    def create_chart(self, summary: dict) -> str:
        return save_chart(summary, "outputs/chart.png")

class MultiAgentSystem:
    def run(self, query: str) -> tuple[str, str]:
        raw = DataGatheringAgent().run(query)
        summary = DataProcessingAgent().summarize(raw)
        chart = VisualizationAgent().create_chart(summary)
        insights = DataProcessingAgent().generate_insights(summary)
        return insights, chart
```

5. Shared Environment Boilerplate

Most assignments used a shared course-level .env file. This keeps keys out of source code and lets multiple projects reuse the same credentials.

```
import os
from pathlib import Path

def load_local_env() -> None:
    for env_path in [Path(__file__).resolve().parents[3] / ".env"]:
        if not env_path.exists():
            continue
        for line in env_path.read_text(encoding="utf-8").splitlines():
            if not line.strip() or "=" not in line:
                continue
            key, value = line.split("=", 1)
            os.environ.setdefault(
                key.strip().strip('"').strip("'").strip(),
                value.strip().strip('"').strip("'").strip(),
            )
    os.environ.setdefault("OPENAI_API_KEY", os.getenv("OPEN_AI_KEY", ""))
```

6. Design Decision Checklist

- **Define the objective:** what is the agent trying to achieve?
- **Collect parameters:** what inputs, preferences, constraints, or keys matter?
- **Choose the pattern:** reflection, tool use, planning, multi-agent, or a combination.
- **Separate responsibilities:** API calls, summarization, reasoning, visualization, and output.
- **Use structured prompts:** specify role, context, expected sections, and constraints.
- **Summarize before prompting:** send compact useful data, not huge raw JSON.
- **Handle errors:** missing keys, API failure, rate limits, empty data, timeouts.
- **Make deliverables visible:** print logs, save outputs, show chart paths, keep screenshots.
- **Reflect on value:** explain why the pattern made the agent better than a simple chatbot.

7. Common Assignment Phrasing

Concept	Useful wording for reflection reports
Reflection	The agent improved its output by critiquing and revising earlier responses.
Tool Use	The external API grounded the response in current, real-world data.
Planning	Structured objectives and parameters helped the model decompose the task into actionable steps.
Multi-Agent	Specialized agents improved clarity by separating data gathering, processing, and presentation.

Fast rule: open chat can stay flexible; specific-goal agents usually benefit from structured data, constraints, expected output shape, and a pattern that matches the workflow.